Foreign Key Columns and Related Queries

Lecture Notes · Game DB Junior

In this lesson

- How to make a foreign key column with **REFERENCES**
- How to insert photos that belong to users
- Two useful queries: **WHERE user_id = ...** and a first look at **JOIN**

Part 1 — Creating the photos table

The photos table looks like the users table, but it also has a **foreign key** column.

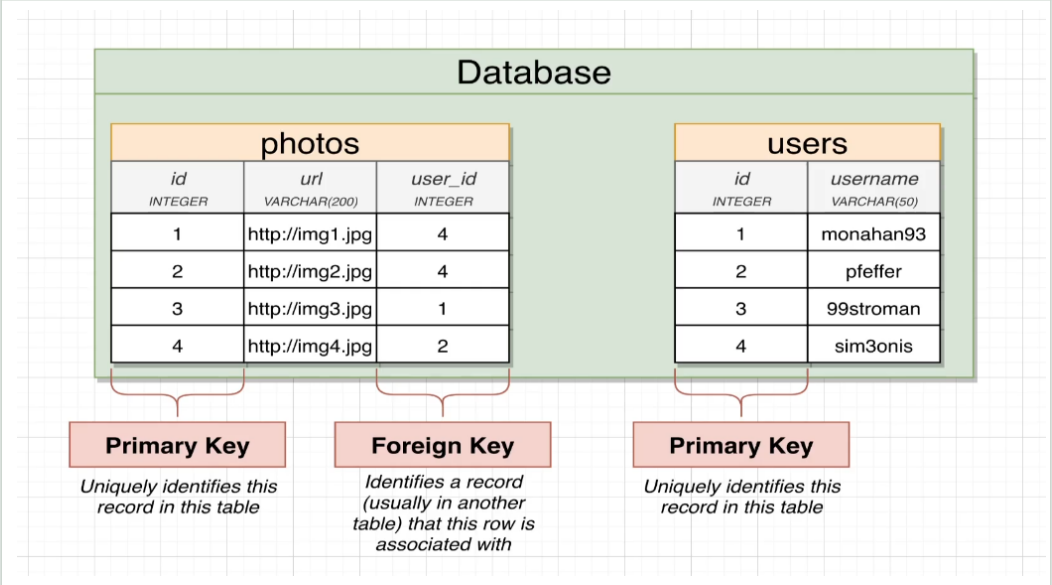


Figure 1 — photos.user_id points at users.id

```
CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    url VARCHAR(200),
    user_id INTEGER REFERENCES users(id)
);
```

Column	What it means
id SERIAL PRIMARY KEY	Made automatically by Postgres. The primary key of the table
url VARCHAR(200)	A link to where the photo is stored. 200 characters, so it can hold a really long URL. We use fake data here
user_id INTEGER	The foreign key. **INTEGER, not SERIAL**: with SERIAL, Postgres would give a value by itself. We want to say exactly which user added the photo

Column	What it means
REFERENCES users(id)	The "magic part". It makes user_id a foreign key: its values point at rows in the **users** table, matching the **id** column

Marking a column as a foreign key gives us some **data consistency**. We will see what this means in a later lesson.

Inserting one photo

We only give the **url** and the **user_id**. The photo id is made automatically. This photo belongs to the user with id 4:

```
INSERT INTO photos (url, user_id)
VALUES ('http://one.jpg', 4);
```

Then **SELECT * FROM photos;** shows the photo with user_id 4.

Why add a foreign key?

- We can write useful queries to get all the photos of a given user.
- We get some **data consistency** (more about this later).

Part 2 — Queries with related data

Adding more photos

Each set of parentheses is one new row. User 1 gets three photos, users 2 and 3 get one each, and user 4 gets one more (two in total). The last line ends with a semicolon.

```
INSERT INTO photos (url, user_id)
VALUES
    ('http://two.jpg', 1),
    ('http://25.jpg', 1),
    ('http://36.jpg', 1),
    ('http://754.jpg', 2),
    ('http://35.jpg', 3),
    ('http://256.jpg', 4);
```

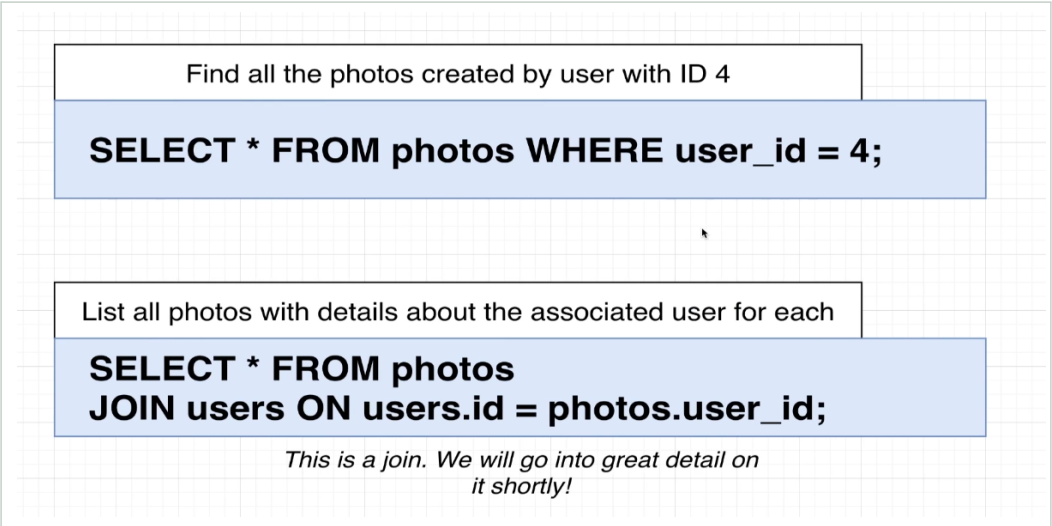


Figure 2 — Two queries we will write often

Query 1: all photos of one user

```
SELECT * FROM photos WHERE user_id = 4;
```

- Think of the **profile page** in Instagram: it lists all the photos one user posted.
- Nothing new here: just SELECT, FROM and WHERE to filter rows.
- User 4 has two photos, so we get **two rows**, both with user_id 4. We could send this data to a front end app to show the photos.

Query 2: a first look at JOIN

Understanding **JOIN** is one of the most challenging things in SQL. For now, we only take a quick look and run it.

```
SELECT url, username FROM photos
JOIN users ON users.id = photos.user_id;
```

- **url** is a column in the photos table. **username** is a column in the users table.
- So JOIN somehow **combines two tables**.
- The result: every photo with the username of the user who added it.
- This is great for a **main feed page**: many photos, each with details about the user who posted it.

Running the code (my practice)

Step 1: create the photos table with a foreign key.

```sql
-- Create a table to store photos
CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    -- VARCHAR(200): link to where the photo is stored (up to 200 characters)
    url VARCHAR(200),
    -- INTEGER, not SERIAL: we give this value ourselves
    -- REFERENCES users(id): the value must match an id in the users table(Foreign Key)
    user_id INTEGER REFERENCES users(id)
);
```

Figure 3 — CREATE TABLE photos with comments

Postgres made the table. Now there are 3 tables: cities, photos, users.

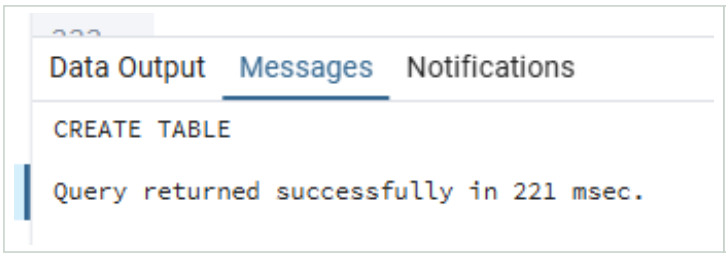


Figure 4 — CREATE TABLE

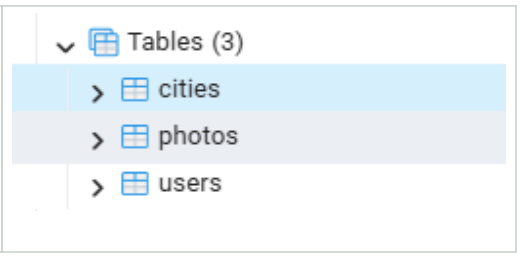


Figure 5 — Tables (3)

Step 2: insert one photo for user 4. The Messages tab shows **INSERT 0 1**.

```sql
-- Add 1 new photo to the photos table
-- We do not give the id; Postgres makes it by itself (SERIAL)
INSERT INTO photos (url, user_id)
VALUES
    -- url: a fake link to the photo
    -- 4: this photo belongs to the user with id 4 (foreign key)
    ('http://one.jpg', 4);
```

Figure 6 — Insert one photo

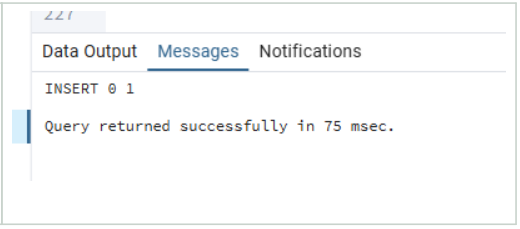


Figure 7 — INSERT 0 1

Step 3: SELECT * FROM photos. The photo got id 1 and has user_id 4.

```sql
SELECT * FROM photos;
```

*Figure 8 — SELECT * FROM photos*

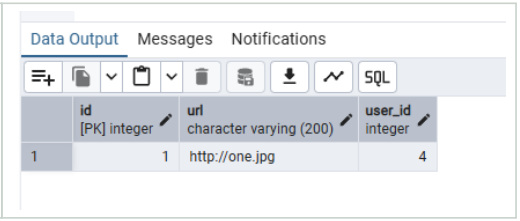


Figure 9 — One photo

Step 4: insert 6 more photos. The Messages tab shows **INSERT 0 6**.

```sql
-- Add 6 more photos to the photos table
-- We do not give the id; Postgres makes it by itself (SERIAL)
-- user_id (foreign key) shows which user each photo belongs to
INSERT INTO photos (url, user_id)
VALUES
    ('http://two.jpg', 1),    -- user 1
    ('http://25.jpg', 1),     -- user 1
    ('http://36.jpg', 1),     -- user 1 (now has 3 photos)
    ('http://754.jpg', 2),    -- user 2 (has 1 photo)
    ('http://35.jpg', 3),     -- user 3 (has 1 photo)
    ('http://256.jpg', 4);    -- user 4 (now has 2 photos, with one.jpg)
                              -- last row ends with ; not ,
```

Figure 10 — Insert 6 photos

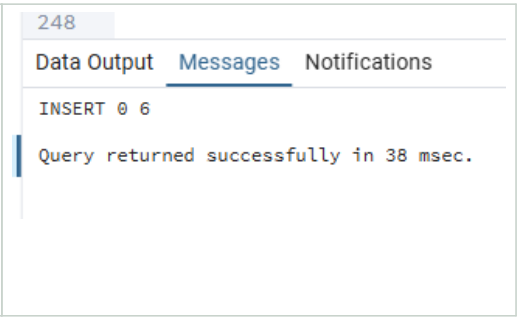
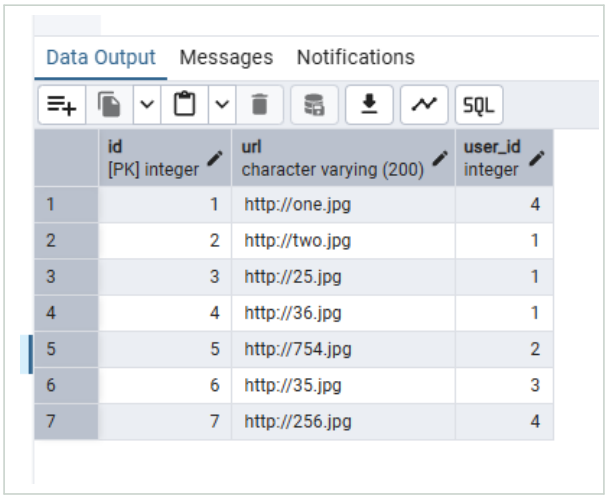


Figure 11 — INSERT 0 6

Now the photos table has 7 photos. User 1 has three, users 2 and 3 have one each, and user 4 has two.



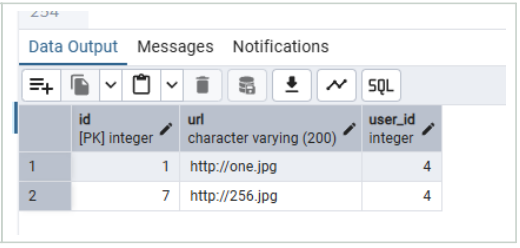
	id [PK] integer	url character varying (200)	user_id integer
1	1	http://one.jpg	4
2	2	http://two.jpg	1
3	3	http://25.jpg	1
4	4	http://36.jpg	1
5	5	http://754.jpg	2
6	6	http://35.jpg	3
7	7	http://256.jpg	4

Figure 12 — All 7 photos

Step 5: all photos of user 4. We get two rows: one.jpg and 256.jpg.

```sql
-- Get all photos that belong to the user with id 4
-- (like a profile page that shows one user's photos)
SELECT * FROM photos
WHERE user_id = 4;
```

Figure 13 — WHERE user_id = 4



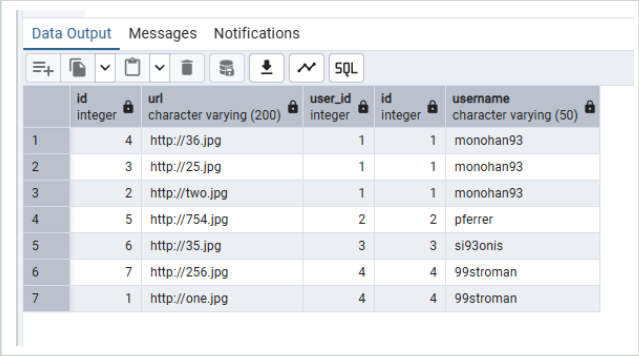
	id [PK] integer	url character varying (200)	user_id integer
1	1	http://one.jpg	4
2	7	http://256.jpg	4

Figure 14 — Two photos

Step 6: JOIN with SELECT *. We get all the columns from both tables: id, url and user_id from photos, and id and username from users.

```sql
-- Get every photo together with the user who added it
-- (like a main feed page)
SELECT * FROM photos      -- * = get all columns from both tables
JOIN users                -- join the users table to photos
ON users.id = photos.user_id;  -- match each photo's user_id with the
                               -- id of the user it belongs to
```

*Figure 15 — JOIN with **



Data Output | Messages | Notifications

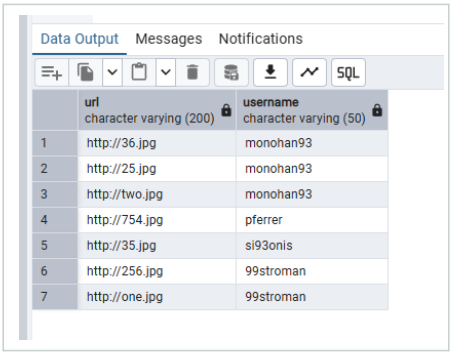
	id integer	url character varying (200)	user_id integer	id integer	username character varying (50)
1	4	http://36.jpg	1	1	monohan93
2	3	http://25.jpg	1	1	monohan93
3	2	http://two.jpg	1	1	monohan93
4	5	http://754.jpg	2	2	pferrer
5	6	http://35.jpg	3	3	si93onis
6	7	http://256.jpg	4	4	99stroman
7	1	http://one.jpg	4	4	99stroman

Figure 16 — Columns from both tables

Step 7: JOIN with only url and username.

```sql
-- Get every photo's link together with the name of the user who added it
-- (like a main feed page)
SELECT url, username      -- url comes from photos, username comes from users
FROM photos               -- start from the photos table
JOIN users                -- join the users table to photos
ON users.id = photos.user_id;  -- match each photo's user_id with the
                               -- id of the user it belongs to
```

Figure 17 — SELECT url, username ... JOIN



Data Output | Messages | Notifications

	url character varying (200)	username character varying (50)
1	http://36.jpg	monohan93
2	http://25.jpg	monohan93
3	http://two.jpg	monohan93
4	http://754.jpg	pferrer
5	http://35.jpg	si93onis
6	http://256.jpg	99stroman
7	http://one.jpg	99stroman

Figure 18 — Each photo with its user's name

> **Note:** in my users table, id 4 is **99stroman** (in the lecture it is sim3onis). So in my JOIN result, one.jpg and 256.jpg belong to 99stroman.

Summary

Idea	Remember
REFERENCES users(id)	Makes a column a foreign key that points at users.id
Foreign key type	INTEGER, not SERIAL. We give the value ourselves
WHERE user_id = 4	All photos of one user (like a profile page)
JOIN users ON users.id = photos.user_id	Combines photos and users (like a main feed). More details later

Next lesson: what foreign key columns really do for us: data consistency.

Sources: Lecture transcripts and lecture images (Figures 1–2); my own practice screenshots (Figures 3–18)